

Spring Boot is a widely used framework for building stand-alone, production-ready Spring applications with minimal configuration. It significantly simplifies the development of Spring-based applications. Here's a detailed list of common Spring Boot interview questions, ranging from basic to advanced, along with comprehensive answers.

Basic Spring Boot Interview Questions

1. What is Spring Boot?

Answer: Spring Boot is an open-source, Java-based framework built on top of the Spring Framework. Its primary goal is to simplify the development of Spring applications by providing a convention-over-configuration approach, auto-configuration, and embedded servers, allowing developers to quickly create stand-alone, production-grade applications with minimal setup.

2. What are the key features of Spring Boot?

Answer: Key features of Spring Boot include:

- **Auto-configuration:** Automatically configures Spring applications based on the dependencies present in the classpath.
- **Starter POMs:** Provides a set of convenient dependency descriptors that simplify dependency management.
- **Embedded Servers:** Comes with embedded servers like Tomcat, Jetty, or Undertow, eliminating the need for external server deployment.
- **Spring Boot Actuator:** Provides production-ready features like monitoring, metrics, and health checks for your application.
- **Spring Boot CLI:** A command-line interface to quickly prototype with Spring.
- **Externalized Configuration:** Allows you to externalize your configuration so you can work with the same application code in different environments.
- **No XML Configuration:** Minimizes the need for verbose XML configurations, favoring annotation-based configurations.

3. What are the advantages of using Spring Boot over traditional Spring?

Answer: Spring Boot offers several advantages:

- **Faster Development:** Reduces development time significantly by providing auto-configuration and opinionated defaults.
- **Simplified Configuration:** Eliminates boilerplate code and XML configuration, making projects easier to set up and maintain.
- **Embedded Servers:** Allows applications to run as standalone JARs, simplifying deployment.
- **Production-Ready Features:** Provides out-of-the-box features for monitoring, health

checks, and metrics through Spring Boot Actuator.

- **Dependency Management:** Starters simplify dependency management by grouping common dependencies.
- **Microservices Friendly:** Highly suitable for developing microservices due to its lightweight and standalone nature.

4. What is @SpringBootApplication annotation?

Answer: @SpringBootApplication is a convenient annotation that combines three commonly used Spring Boot annotations:

- @Configuration: Tags the class as a source of bean definitions for the application context.
- @EnableAutoConfiguration: Enables Spring Boot's auto-configuration mechanism, which automatically configures your application based on the classpath dependencies.
- @ComponentScan: Enables component scanning on the package where the application class is located (and its sub-packages), allowing Spring to discover and register components (e.g., @Controller, @Service, @Repository, @Component).

5. What are Spring Boot Starters? Provide some examples.

Answer: Spring Boot Starters are a set of convenient dependency descriptors that you can include in your application's pom.xml (for Maven) or build.gradle (for Gradle). They provide a curated set of dependencies that are commonly used together for a specific functionality, simplifying dependency management and ensuring compatibility.

Examples:

- spring-boot-starter-web: For building web applications, including Spring MVC, embedded Tomcat, and RESTful capabilities.
- spring-boot-starter-data-jpa: For using Spring Data JPA with Hibernate for database interactions.
- spring-boot-starter-test: For testing Spring Boot applications, including JUnit, Mockito, and Spring Test.
- spring-boot-starter-security: For adding security features to your application.

6. What is Spring Boot Auto-configuration? How does it work?

Answer: Spring Boot Auto-configuration is a core feature that automatically configures your Spring application based on the JAR dependencies you've added to your project. It intelligently guesses and sets up common configurations, reducing the need for explicit XML or Java-based configurations.

It works by:

- **Conditional Annotations:** Spring Boot uses various @Conditional annotations (e.g.,

@ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty) to determine whether a particular auto-configuration class should be applied.

- **spring.factories:** Auto-configuration classes are registered in META-INF/spring.factories files within the Spring Boot starter JARs. When your application starts, Spring Boot scans these files to find and apply relevant auto-configurations.

7. How do you create a Spring Boot application?

Answer: You can create a Spring Boot application in several ways:

- **Spring Initializr (start.spring.io):** This is the most common and recommended way. It's a web-based tool that allows you to select project metadata, dependencies, and then generates a downloadable zip file with a ready-to-use Spring Boot project structure.
- **Spring Boot CLI:** You can use the Spring Boot Command Line Interface to quickly prototype applications with Groovy.
- **Manually:** You can set up a Maven or Gradle project and add the necessary Spring Boot dependencies manually.

8. What is application.properties or application.yml file used for?

Answer: These files are used for externalized configuration in Spring Boot applications. They allow you to define various properties like server port, database connection details, logging levels, and other application-specific settings. Spring Boot automatically loads properties from these files at startup, and you can access them using @Value annotation or @ConfigurationProperties. application.yml is often preferred for its cleaner, hierarchical structure.

9. What is Spring Boot Actuator? What are its common endpoints?

Answer: Spring Boot Actuator provides production-ready features to monitor and manage your application. It exposes various endpoints that allow you to inspect the application's internal state, health, metrics, environment, and more.

Common endpoints include:

- **/health:** Shows application health information.
- **/info:** Displays arbitrary application info.
- **/metrics:** Provides various application metrics (e.g., memory usage, HTTP requests).
- **/env:** Shows the current environment properties.
- **/beans:** Lists all Spring beans in the application context.
- **/shutdown:** Allows graceful shutdown of the application (requires explicit enabling).

10. What is the difference between @Controller and @RestController?

Answer:

- **@Controller:** This annotation is typically used in Spring MVC applications to mark a class

as a Spring web controller. It's usually combined with `@ResponseBody` on individual methods if you want to return data directly as JSON/XML instead of a view.

- **@RestController**: This is a specialized version of `@Controller` that combines `@Controller` and `@ResponseBody`. It's primarily used for building RESTful web services, as it automatically serializes the return values of methods into the HTTP response body (e.g., JSON or XML). This eliminates the need to add `@ResponseBody` to every method.

Intermediate Spring Boot Interview Questions

11. Explain Dependency Injection (DI) and Inversion of Control (IoC) in the context of Spring Boot.

Answer:

- **Inversion of Control (IoC)**: IoC is a design principle where the control of object creation and lifecycle management is transferred from the application code to a container (the Spring IoC container). Instead of your code creating dependent objects, the container creates and manages them.
- **Dependency Injection (DI)**: DI is a specific implementation of IoC. It's a pattern where the Spring IoC container "injects" (provides) the dependencies (collaborating objects) into a class, rather than the class creating or looking up its dependencies. This promotes loose coupling and testability. In Spring Boot, DI is commonly achieved through annotations like `@Autowired`, constructor injection, or setter injection.

12. How does Spring Boot handle externalized configuration?

Answer: Spring Boot supports externalized configuration, allowing you to run the same application code in different environments (development, testing, production) with different configurations. It provides a flexible way to define properties, with precedence rules.

Sources of external configuration (in order of precedence, higher overrides lower):

1. Command-line arguments
2. `SpringApplication.setDefaultProperties`
3. OS environment variables
4. Java System properties (`-Dkey=value`)
5. Application properties loaded from `application.properties` or `application.yml` files (inside the JAR, then outside the JAR).
6. Profile-specific properties (`application-{profile}.properties`)
7. `@PropertySource` annotations
8. `application.properties` in the classpath.

You can access these properties using:

- `@Value("${property.name}")`
- `@ConfigurationProperties` for type-safe configuration.

- Environment abstraction.

13. What are Spring Boot Profiles? How do you use them?

Answer: Spring Boot Profiles provide a way to segregate configuration for different environments (e.g., dev, qa, prod). This allows you to have different bean definitions, property values, or even entire configurations active only in specific environments.

How to use them:

- **Defining Profiles:**
 - application-{profile}.properties or application-{profile}.yml files: For example, application-dev.properties for development-specific properties.
 - @Profile annotation: On @Configuration classes or @Component beans to make them active only for certain profiles.
- **Activating Profiles:**
 - spring.profiles.active property: Set it in application.properties (e.g., spring.profiles.active=dev) or as a command-line argument (--spring.profiles.active=dev).
 - Environment variable SPRING_PROFILES_ACTIVE.

14. Explain @Autowired and its different types of injection.

Answer: @Autowired is a Spring annotation used for automatic dependency injection. Spring automatically resolves and injects beans into the annotated field, constructor, or setter method.

Types of injection:

- **Field Injection (Least Recommended):**

```
Java
@Autowired
private MyService myService;
```

- **Pros:** Concise.
- **Cons:** Makes unit testing difficult (requires reflection), hides dependencies, and can lead to circular dependencies being harder to spot.

- **Setter Injection:**

```
Java
private MyService myService;

@Autowired
public void setMyService(MyService myService) {
    this.myService = myService;
}
```

- **Pros:** Allows for optional dependencies (by making the setter optional), mutable.
- **Cons:** Can still be modified after creation, and dependencies aren't immediately clear from the constructor.
- **Constructor Injection (Most Recommended):**

```
Java
private final MyService myService;

public MyController(MyService myService) {
    this.myService = myService;
}
```

 - **Pros:**
 - **Immutability:** Ensures that dependencies are set at object creation and cannot be changed later.
 - **Testability:** Easier to unit test as dependencies are explicitly passed in.
 - **Clarity:** Clearly shows required dependencies.
 - **Circular Dependency Detection:** Spring can detect circular dependencies at startup, preventing runtime issues.
 - **Cons:** Can lead to long constructors if there are many dependencies (though this often indicates a class with too many responsibilities).

15. What is Spring Data JPA? How does it simplify data access?

Answer: Spring Data JPA is a part of the larger Spring Data project that simplifies data access and manipulation using JPA (Java Persistence API). It significantly reduces the amount of boilerplate code needed for CRUD (Create, Read, Update, Delete) operations.

How it simplifies data access:

- **Repository Abstraction:** It provides a Repository interface and its sub-interfaces (e.g., JpaRepository, CrudRepository) that define common data access methods. You just need to extend these interfaces, and Spring Data JPA automatically provides the implementation at runtime.
- **Derived Query Methods:** You can define methods in your repository interfaces with specific naming conventions (e.g., findByLastNameAndFirstName(String lastName, String firstName)), and Spring Data JPA will automatically generate the corresponding JPA query.
- **Custom Queries:** For more complex queries, you can use the @Query annotation with JPQL or native SQL.
- **Pagination and Sorting:** Built-in support for pagination and sorting using Pageable and Sort objects.

16. How do you handle exceptions in Spring Boot REST APIs?

Answer: Spring Boot provides several ways to handle exceptions in REST APIs:

- **@ExceptionHandler:** Define methods within a @Controller or @RestController to handle specific exceptions for that controller.
- **@ControllerAdvice and @RestControllerAdvice:** These annotations allow you to define global exception handlers across multiple controllers. A class annotated with @ControllerAdvice (or @RestControllerAdvice which implies @ResponseBody) can contain @ExceptionHandler methods to centralize error handling.
- **ErrorController:** Spring Boot provides a default ErrorController to handle all errors that are not handled by custom exception handlers. You can customize this by implementing your own ErrorController.
- **ResponseEntityExceptionHandler:** Spring provides ResponseEntityExceptionHandler which is a convenient base class for @ControllerAdvice implementations that handles standard Spring MVC exceptions and allows you to customize the response.

Example with @RestControllerAdvice:

Java

@RestControllerAdvice

public class GlobalExceptionHandler {

@ExceptionHandler(ResourceNotFoundException.class)

@ResponseStatus(HttpStatus.NOT_FOUND)

public ErrorResponse handleResourceNotFoundException(ResourceNotFoundException ex) {

return new ErrorResponse(HttpStatus.NOT_FOUND.value(), ex.getMessage());

}

@ExceptionHandler(MethodArgumentNotValidException.class)

@ResponseStatus(HttpStatus.BAD_REQUEST)

public Map<String, String> handleValidationExceptions(MethodArgumentNotValidException ex) {

Map<String, String> errors = new HashMap<>();

ex.getBindingResult().getAllErrors().forEach((error) -> {

String fieldName = ((FieldError) error).getField();

String errorMessage = error.getDefaultMessage();

errors.put(fieldName, errorMessage);

});

return errors;

}

// Generic exception handler


```

@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ErrorResponse handleGenericException(Exception ex) {
    return new ErrorResponse(HttpStatus.INTERNAL_SERVER_ERROR.value(), "An unexpected
error occurred.");
}
}

```

17. What is Spring Boot DevTools? How does it help in development?

Answer: Spring Boot DevTools is a module that provides additional development-time features to enhance the developer experience. It aims to reduce development time and improve productivity.

Key features:

- **Automatic Restart:** When you make changes to your code, DevTools automatically restarts the application.
- **LiveReload:** Integrates with browser extensions to automatically refresh the browser when changes are detected (e.g., HTML, CSS, JavaScript).
- **Global Settings:** Allows you to configure global settings for all your Spring Boot projects.
- **Disable Template Caching:** Disables caching for template engines (like Thymeleaf, Freemarker) during development, so changes are immediately visible.
- **Embedded Database Support:** Provides special configurations for embedded databases like H2, HSQL, and Derby, allowing data to be reloaded on application restart.

18. How do you implement logging in Spring Boot?

Answer: Spring Boot provides robust logging support out-of-the-box. It uses commons-logging internally and delegates to various logging frameworks. By default, it uses Logback with SLF4J.

- **Default Configuration:** Spring Boot pre-configures Logback to log to the console.
- **application.properties / application.yml:** You can configure logging levels for different packages or classes using properties like:

Properties

logging.level.root=INFO

logging.level.com.example.myapp=DEBUG

logging.file.name=myapp.log

- **logback-spring.xml:** For more advanced logging configurations (e.g., custom appenders, log rotation), you can provide a logback-spring.xml file in the src/main/resources folder. Spring Boot will pick it up automatically.
- **SLF4J Logger:** You typically use org.slf4j.Logger for logging within your code:

Java


```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Service
public class MyService {
    private static final Logger logger = LoggerFactory.getLogger(MyService.class);

    public void doSomething() {
        logger.info("Doing something important...");
        logger.debug("Debug message: variable = {}", someVariable);
        // ...
    }
}
```

19. What is the role of CommandLineRunner and ApplicationRunner interfaces in Spring Boot?

Answer: Both CommandLineRunner and ApplicationRunner interfaces are functional interfaces used to execute specific code after the Spring Boot application context has been loaded and initialized. They are useful for running tasks that need to be performed once the application starts up, such as data initialization, pre-computation, or background tasks.

- **CommandLineRunner:** The run method in CommandLineRunner accepts command-line arguments as a String[].

```
Java
@Component
public class MyCommandLineRunner implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Application started with arguments: " + Arrays.toString(args));
        // Perform startup tasks
    }
}
```

- **ApplicationRunner:** The run method in ApplicationRunner accepts command-line arguments as an ApplicationArguments object, which provides a more structured way to access options and non-option arguments.

```
Java
@Component
public class MyApplicationRunner implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) throws Exception {
        System.out.println("Non-option arguments: " + args.getNonOptionArgs());
    }
}
```

```

    System.out.println("Option arguments: " + args.getOptionNames());
    // Perform startup tasks
}
}

```

You can have multiple implementations of these interfaces, and you can control their execution order using `@Order` annotation.

20. How do you secure a Spring Boot application?

Answer: Spring Boot provides excellent integration with Spring Security to secure applications. Common ways to secure a Spring Boot application include:

- **Spring Security Dependency:** Add `spring-boot-starter-security` to your `pom.xml`.
- **Basic Authentication:** By default, adding the security starter enables basic authentication with a generated password (logged to the console).
- **Form-based Login:** Configure a `WebSecurityConfigurerAdapter` (or more recently, a `SecurityFilterChain` bean) to enable form-based login, define user details, and authorize requests.
- **OAuth2 / JWT:** For more advanced scenarios like securing REST APIs, you can integrate OAuth2 or JWT (JSON Web Tokens) for token-based authentication and authorization. Spring Security provides good support for both.
- **Method-Level Security:** Use annotations like `@PreAuthorize`, `@PostAuthorize`, `@Secured`, `@RolesAllowed` to secure individual methods.
- **CORS (Cross-Origin Resource Sharing):** Configure CORS policies to allow or restrict requests from different origins.

Advanced Spring Boot Interview Questions

21. Explain the concept of @Conditional annotations in Spring Boot.

Answer: `@Conditional` annotations are a powerful mechanism in Spring Boot that allows for conditional bean registration and configuration. They enable components to be included or excluded based on certain conditions at runtime, making auto-configuration possible.

Examples of `@Conditional` annotations:

- `@ConditionalOnClass`: The configuration class or bean is only included if the specified class is on the classpath.
- `@ConditionalOnMissingClass`: The configuration class or bean is only included if the specified class is *not* on the classpath.
- `@ConditionalOnBean`: The configuration class or bean is only included if the specified bean type exists in the Spring application context.
- `@ConditionalOnMissingBean`: The configuration class or bean is only included if the

specified bean type *does not* exist.

- `@ConditionalOnProperty`: The configuration class or bean is included based on the presence or value of a specific Spring Boot property.
- `@ConditionalOnWebApplication`: The configuration class or bean is included only if the application is a web application (e.g., Servlet or Reactive).
- `@ConditionalOnNotWebApplication`: The configuration class or bean is included only if the application is *not* a web application.

These annotations allow Spring Boot to dynamically configure the application based on what dependencies are available and what properties are set, providing its "opinionated defaults" and auto-configuration capabilities.

22. How do you create a custom Spring Boot Starter?

Answer: Creating a custom Spring Boot Starter involves packaging a set of common dependencies and auto-configuration classes into a reusable module. This allows other projects to easily pull in your functionality with a single dependency.

Steps:

1. **Create a Maven/Gradle project:** Name it something like `my-custom-spring-boot-starter`.
2. **Add spring-boot-autoconfigure dependency:** This provides the auto-configuration capabilities.
3. **Define Auto-configuration Classes:** Create a class annotated with `@Configuration` that defines the beans your starter will provide. Use `@Conditional` annotations to make these beans conditional based on user preferences or the classpath.

Java

`@Configuration`

`@ConditionalOnClass(MyService.class)` // Only if MyService is on classpath

`@EnableConfigurationProperties(MyServiceProperties.class)`

`public class MyServiceAutoConfiguration {`

`private final MyServiceProperties properties;`

`public MyServiceAutoConfiguration(MyServiceProperties properties) {`

`this.properties = properties;`

`}`

`@Bean`

`@ConditionalOnMissingBean`

`public MyService myService() {`

`return new MyService(properties.getMessage());`

`}`

```
}
```

4. **Register Auto-configuration:** Create a META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (Spring Boot 2.7+) or META-INF/spring.factories (older versions) file in your starter's src/main/resources directory. In this file, list your auto-configuration class:
`org.springframework.boot.autoconfigure.EnableAutoConfiguration=\ncom.example.starter.MyServiceAutoConfiguration`

5. **Add spring-boot-starter to your client project:** Other projects can then simply add your custom starter as a dependency:

XML

```
<dependency>\n  <groupId>com.example</groupId>\n  <artifactId>my-custom-spring-boot-starter</artifactId>\n  <version>1.0.0</version>\n</dependency>
```

23. Discuss the difference between WebClient and RestTemplate in Spring Boot. When would you use one over the other?

Answer: Both WebClient and RestTemplate are used for making HTTP requests in Spring. However, they belong to different paradigms and are suited for different use cases.

- **RestTemplate (Synchronous and Blocking):**
 - **Nature:** Synchronous and blocking client. When a request is made, the calling thread is blocked until the response is received.
 - **Origin:** Part of Spring's spring-web module, older and widely used.
 - **Concurrency:** Handles concurrency by creating a new thread for each request.
 - **Error Handling:** Throws exceptions for HTTP errors.
 - **Use Cases:** Suitable for simpler, traditional synchronous applications, or when interacting with older blocking APIs.
 - **Deprecation:** In Spring 5, RestTemplate is in maintenance mode, and WebClient is the recommended alternative.
- **WebClient (Asynchronous and Non-Blocking / Reactive):**
 - **Nature:** Asynchronous and non-blocking client, built on Project Reactor. It returns Mono (for 0 or 1 element) or Flux (for 0 to N elements) as reactive streams.
 - **Origin:** Introduced in Spring 5 as part of Spring WebFlux, designed for reactive programming.
 - **Concurrency:** Achieves concurrency with a small number of threads by using an event-loop model.
 - **Error Handling:** Uses reactive error handling mechanisms (e.g., onErrorResume, onErrorReturn).

- **Use Cases:**
 - Building reactive microservices or applications.
 - When high concurrency and low latency are critical.
 - Integrating with other reactive systems.
 - When you want to avoid thread-per-request overhead.

When to use which:

- Use WebClient for new projects, especially if you're building reactive applications or microservices, or if performance under high concurrency is a priority.
- Continue using RestTemplate in existing legacy projects if migrating to WebClient is not feasible or if the synchronous nature is acceptable for the specific use case. However, for any new HTTP client needs in Spring, WebClient is the recommended path forward.

24. How do you implement distributed transactions in Spring Boot microservices?

Answer: Implementing distributed transactions (transactions spanning across multiple microservices) is a complex challenge. Traditional two-phase commit (2PC) is often not suitable for microservices architectures due to its blocking nature and tight coupling.

Common approaches for distributed transactions in microservices:

1. **Saga Pattern:** This is the most common approach. A saga is a sequence of local transactions, where each local transaction updates data within a single service and publishes an event that triggers the next local transaction in the saga. If a local transaction fails, the saga executes compensating transactions to undo the changes made by previous successful transactions.
 - **Choreography-based Saga:** Services communicate directly via events, without a central orchestrator. Each service listens for events and publishes its own events.
 - **Orchestration-based Saga:** A central orchestrator (a separate service or a component within one of the services) tells each participant service what local transaction to execute.
 - **Spring Boot with Saga:** You can implement sagas using message brokers (e.g., Kafka, RabbitMQ) for event communication, and frameworks like Axon Framework or Eventuate Tram for managing saga state and compensating actions.
2. **Idempotent Consumers:** Design your services to be idempotent, meaning that applying the same operation multiple times will produce the same result as applying it once. This helps in handling retries and potential duplicate messages in a distributed system.
3. **Compensating Transactions:** As part of the Saga pattern, compensating transactions are crucial. These are operations that undo the effects of a previous transaction.
4. **Eventual Consistency:** Often, for non-critical scenarios, an eventual consistency model is adopted, where data across services might be temporarily inconsistent but eventually converges.
5. **Database per Service:** Each microservice has its own database, preventing direct

database-level transactions across services and forcing communication via APIs and events.

Spring Boot itself doesn't directly solve distributed transactions, but it provides the foundational tools (like Spring Data JPA, Spring Messaging, Spring Cloud Stream) to build solutions based on patterns like Saga.

25. Explain the Circuit Breaker pattern and how to implement it in Spring Boot.

Answer: The Circuit Breaker pattern is a fault tolerance technique used in distributed systems to prevent cascading failures when a service calls another service that is failing or exhibiting high latency. It works by monitoring calls to an external service, and if the failure rate exceeds a threshold, it "breaks" the circuit, preventing further calls to the failing service.

States of a Circuit Breaker:

1. **Closed:** The default state. Requests are passed to the service. If failures exceed a threshold, it transitions to **Open**.
2. **Open:** Requests to the service are immediately rejected, and an alternative (fallback) is executed. After a specified timeout, it transitions to **Half-Open**.
3. **Half-Open:** A limited number of test requests are allowed to pass through to the service. If these requests succeed, it transitions back to **Closed**. If they fail, it transitions back to **Open**.

Implementation in Spring Boot:

Spring Cloud Netflix Hystrix was a popular choice, but it's now in maintenance mode. The recommended alternative for implementing Circuit Breaker in Spring Boot is Resilience4j.

Steps with Resilience4j:

1. **Add Dependency:**

```
XML
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version> </dependency>
```

2. **Configure Circuit Breaker in application.yml:**

```
YAML
resilience4j.circuitbreaker:
  instances:
    myServiceCircuitBreaker:
      registerHealthIndicator: true
      failureRateThreshold: 50 # 50% failures
      minimumNumberOfCalls: 5 # Min calls before evaluating failure rate
```

```
automaticTransitionFromOpenToHalfOpenEnabled: true
waitDurationInOpenState: 5s # Wait 5 seconds in open state
permittedNumberOfCallsInHalfOpenState: 3 # 3 calls allowed in half-open
```

3. Apply @CircuitBreaker Annotation:

Java

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class ExternalService {
```

```
    private static final String SERVICE_NAME = "myServiceCircuitBreaker"; // Matches instance
    name
```

```
    @CircuitBreaker(name = SERVICE_NAME, fallbackMethod = "fallbackForExternalService")
```

```
    public String callExternalService() {
```

```
        // Simulate calling an external service
```

```
        if (Math.random() > 0.5) {
```

```
            throw new RuntimeException("External service failed!");
```

```
        }
```

```
        return "External service response";
```

```
    }
```

```
    public String fallbackForExternalService(Throwable t) {
```

```
        return "Fallback response: External service is currently unavailable. Error: " +
```

```
t.getMessage();
```

```
    }
```

```
}
```

When callExternalService() fails repeatedly, the circuit will open, and subsequent calls will immediately trigger fallbackForExternalService() without attempting to call the actual service.

This detailed set of questions and answers covers a broad range of Spring Boot concepts, from fundamental features to more advanced topics like distributed transactions and fault tolerance. Remember to emphasize practical experience and be able to discuss trade-offs and design decisions during an interview.